

aligned_accessor: An mdspan accessor expressing pointer overalignment

Document #: P2897R1
Date: 2023-10-12
Project: Programming Language C++ LEWG
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Nicolas Morales
<nmmoral@sandia.gov>
Christian Trott
<crtrott@sandia.gov>

Contents

- [1 Authors](#)
- [2 Revision history](#)
- [3 Purpose of this paper](#)
- [4 Key features](#)
- [5 Design discussion](#)
 - [5.1 The accessor is not nestable](#)
 - [5.2 Explicit constructor from default_accessor](#)
 - [5.3 We do not define an alias for aligned mdspan](#)
 - [5.4 mdspan construction safety](#)
- [6 Implementation](#)
- [7 Example](#)
- [8 Wording](#)
 - [8.1 Add subsection !\[\]\(38441ceaa711016e0bf2ad46ad394ff4_img.jpg\) \[mdspan.accessor.aligned\] with the following](#)
 - [8.2 Members \[mdspan.accessor.aligned.members\]](#)

§ 1 Authors

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)
- Damien Lebrun-Grandie (lebrungrandt@ornl.gov) (Oak Ridge National Laboratory)
- Nicolas Morales (nmmoral@sandia.gov) (Sandia National Laboratories)
- Christian Trott (crtrott@sandia.gov) (Sandia National Laboratories)

§ 2 Revision history

- Revision 0 (pre-Varna) to be submitted 2023-05-19
- Revision 1 (pre-Kona) to be submitted 2023-10-15
 - Implement changes requested by LEWG review on 2023/10/10
 - Change `gcd` converting constructor `Constraint` to a `Mandate`
 - Add Example in the wording section that uses `is_sufficiently_aligned` to check the pointer overalignment precondition
 - Add Example in the wording section that uses `aligned_alloc` to create an overaligned allocation, to show that `aligned_accessor` exists as part of a system
 - Add an explicit constructor from `default_accessor`, so that users can type `aligned_mdspan y{x}` instead of `aligned_mdspan y{x.data_handle(), x.mapping() }`. Add an explanation in the design discussion section.
 - Implement other wording changes
 - Add to `aligned_accessor`'s Mandates that `byte_alignment >= alignof(ElementType)` is true. This prevents construction of an invalid `aligned_accessor` object.
 - Add more design discussion based on LEWG review on 2023/10/10
 - Explain why we do not include an `aligned_mdspan` alias
 - Explain `aligned_accessor` construction safety

§ 3 Purpose of this paper

We propose adding `aligned_accessor` to the C++ Standard Library. This class template is an `mdspan` accessor policy that uses `assume_aligned` to decorate pointer access. We think it belongs in the Standard Library for two reasons. First, it would serve as a common vocabulary type for interfaces that take `mdspan` to declare their minimum alignment requirements. Second, it extends to `mdspan` accesses the optimizations that compilers can perform to pointers decorated with `assume_aligned`.

`aligned_accessor` is analogous to the various `atomic_accessor_*` templates proposed by P2689. Both that proposal and this one start with a Standard Library feature that operates on a “raw” pointer (`assume_aligned` or the various `atomic_ref*` templates), and then propose an `mdspan` accessor policy that straightforwardly wraps the lower-level feature.

We had originally written `aligned_accessor` as an example in P2642, which proposes “padded” `mdspan` layouts. We realized that `aligned_accessor` was more generally applicable and that standardization would help the padded layouts proposed by P2642 reach their maximum value.

§ 4 Key features

- `offset_policy` is `default_accessor`
- `data_handle_type` is `ElementType*`
- Constructor permits conversion
 - from `nonconst` to `const ElementType`, and
 - from more overalignment to less overalignment
- `is_sufficiently_aligned` checks pointer alignment

The `offset_policy` alias is `default_accessor<ElementType>`, because even if a pointer `p` is aligned, `p + i` might not be.

The `data_handle_type` alias is `ElementType*`. It needs no further adornment, because alignment is asserted at the point of access, namely in the access function. Some implementations might have an easier time optimizing if they also apply some implementation-specific attribute to `data_handle_type` itself. Examples of such attributes include `__declspec(align_value(byte_alignment))` and `__attribute__((align_value(byte_alignment)))`. However, these attributes should not apply to the result of `offset`, for the same reason that `offset_policy` is `default_accessor` and not `aligned_accessor`.

The constructor is analogous to `default_accessor`’s constructor, in that it exists to permit conversion from `nonconst element_type` to `const element_type`. The constructor of `aligned_accessor` additionally permits conversion from more overalignment to less overalignment – something that we expect users may need to do. For example, users may start with `aligned_accessor<float, 128>`, because their allocation function promises 128-byte alignment. However, they may then need to call a function that takes an `mdspan` with `aligned_accessor<float, 32>`, which declares the function’s intent to use 8-wide SIMD of `float`.

The `is_sufficiently_aligned` function checks whether a pointer has sufficient alignment to be used correctly with the class. This makes it easier for users to check

preconditions, without needing to know how to cast a pointer to an integer of the correct size and signedness.

§ 5 Design discussion

§ 5.1 The accessor is not nestable

We considered making `aligned_accessor` “wrap” any accessor type meeting the right requirements. For example, `aligned_accessor` could take the inner accessor as a template parameter, store an instance of it, and dispatch to its member functions. That would give users a way to apply multiple accessor “attributes” to their data handle, such as atomic access (see P2689) and overalignment.

We decided against this approach for three reasons. First, we would have no way to validate that the user’s accessor type has the correct behavior. We could check that their accessor’s `data_handle_type` is a pointer type, but we could not check that their accessor’s access function actually dereferences the pointer. For instance, `access` might instead interpret the data handle as a file handle or a key into a distributed data store.

Second, even if the inner accessor’s access function actually did return the result of dereferencing the pointer, the outer access function might not be able to recover the effects of the inner access function, because `access` computes a reference, not a pointer. For example, our `aligned_accessor`’s access function can only apply `assume_aligned` to the pointer; knowledge of overalignment at compile time “lost” with the dereference at element `i`.

Third, any way (not just this one) of nesting two generic accessors raises the question of order dependence. Even if it were possible to apply the effects of both the inner and outer accessors’ access functions in sequence, it might be unpleasantly surprising to users if the effects depended on the order of nesting. A similar question came up in the “properties” proposal P0900, which we quote here.

Practically speaking, it would be considered a best practice of a high-quality implementation to ensure that a property’s implementation of `properties::element_type_t` (and other traits) are invariant with respect to ordering with other known properties (such as those in the standard library), but with this approach it would be impossible to make that guarantee formal, particularly with respect to other vendor-defined and user-defined properties unknown to the property implementer.

For these reasons, we have made `aligned_accessor` stand-alone, instead of having it modify another user-provided accessor.

§ 5.2 Explicit constructor from `default_accessor`

LEWG's 2023/10/10 review of R0 pointed out that in R0, `aligned_accessor` lacks an explicit constructor from `default_accessor`. Having that constructor would make it easier for users to create an aligned `mdspan` from an unaligned `mdspan`. Making it explicit would prevent implicit conversion. Thus, we have decided to add this explicit constructor in R1.

Without the explicit constructor, users must do this to convert from nonaligned to aligned `mdspan`.

```
void compute_with_aligned(
    std::mdspan<float, std::dextent<int, 2>, std::layout_left>
matrix)
{
    const std::size_t byte_alignment = 4 * alignof(float);
    using aligned_matrix_t = std::mdspan<float, std::dextent<int,
2>,
        std::layout_left, std::aligned_accessor<float,
byte_alignment>>>;

    aligned_matrix_t aligned_matrix{matrix.data_handle(),
matrix.mapping()};
    // ... use aligned_matrix ...
}
```

Another option would be to use constructor template argument deduction (CTAD), as in the following.

```
void compute_with_aligned(
    std::mdspan<float, std::dextent<int, 2>, std::layout_left>
matrix)
{
    const std::size_t byte_alignment = 4 * alignof(float);

    std::mdspan aligned_matrix{matrix.data_handle(),
matrix.mapping(),
    std::aligned_accessor<float, byte_alignment>{}};
    // ... use aligned_matrix ...
}
```

However, `mdspan` users commonly define their own type aliases for `mdspan`, with application-specific names that make code more self-documenting. The `aligned_matrix_t` definition above is an example. Such users would not normally rely on CTAD for conversion from a less specific to a more specific `mdspan`.

Adding an explicit constructor from `default_accessor` lets users get the same effect more concisely.

```
void compute_with_aligned(std::mdspan<float, std::dextent<int, 2>,
std::layout_left> matrix)
{
    const std::size_t byte_alignment = 4 * alignof(float);
    using aligned_mdspan = std::mdspan<float, std::dextent<int, 2>,
        std::layout_left, std::aligned_accessor<float,
```

```
byte_alignment>>;

    aligned_mdspan aligned_matrix{matrix};
    // ... use aligned_matrix ...
}
```

The explicit constructor does not decrease safety, in the sense that users were always allowed to convert from an `mdspan` with `default_accessor` to an `mdspan` with `aligned_accessor`. Before, users could perform this conversion by typing the following.

```
aligned_matrix_t aligned_matrix{matrix.data_handle(),
matrix.mapping()};
```

Now, users can do the same thing with fewer characters.

```
aligned_matrix_t aligned_matrix{matrix};
```

§ 5.3 We do not define an alias for aligned mdspan

In LEWG’s 2023/10/10 review of R0, participants observed that this proposal’s examples define an example-specific type alias for `mdspan` with `aligned_accessor`. They asked whether our proposal should include a *standard* alias `aligned_mdspan`. We do not *object* to such an alias, but we do not find it very useful, for the following reasons.

1. Users of `mdspan` commonly define their own type aliases whose names have meaningful names for their applications.
2. It would not save much typing.

Examples may define aliases to make them more concise. One of them in this proposal defines the following alias for an `mdspan` of `float` with alignment `byte_alignment`.

```
template<size_t byte_alignment>
using aligned_mdspan = std::mdspan<float, std::dextents<int, 1>,
std::layout_right, std::aligned_accessor<float,
byte_alignment>>>;
```

This lets the example use `aligned_mdspan<32>` and `aligned_mdspan<16>`.

The above alias is specific to a particular example. A *general* version of alias would look like this.

```
template<class ElementType, class Extents, class Layout, size_t
byte_alignment>
using aligned_mdspan = std::mdspan<ElementType, Extents, Layout,
std::aligned_accessor<ElementType, byte_alignment>>>;
```

This alias would save *some* typing. However, `mdspan` “power users” rarely type out all the template arguments. First, they can rely on CTAD to create `mdspans`, and `auto` to return them. Second, users commonly already define their own aliases whose names have an

application-specific meaning. They define these aliases *once* and use them throughout the application. For instance, users might define the following.

```
template<class ElementType>
using vector_t = std::mdspan<ElementType, std::dextents<int, 1>,
std::layout_left>;
template<class ElementType>
using matrix_t = std::mdspan<ElementType, std::dextents<int, 2>,
std::layout_left>;

template<class ElementType, size_t byte_alignment>
using aligned_vector_t = std::mdspan<ElementType,
std::dextents<int, 1>, std::layout_left,
std::aligned_accessor<ElementType, byte_alignment>>;
template<class ElementType, size_t byte_alignment>
using aligned_matrix_t = std::mdspan<ElementType,
std::dextents<int, 2>, std::layout_left,
std::aligned_accessor<ElementType, byte_alignment>>;
```

Such users may never type the characters “mdspan” again. For this reason, while we do not object to an `aligned_mdspan` alias, we do not find the proliferation of aliases particularly ergonomic.

§ 5.4 mdspan construction safety

LEWG’s 2023/10/10 review of R0 expressed concern that `mdspan`’s constructor has no way to check `aligned_accessor`’s alignment requirements. Users can call `aligned_accessor`’s `is_sufficiently_aligned(p)` static member function with a pointer `p` to check this themselves, before constructing the `mdspan`. However, `mdspan`’s constructor generally has no way to check whether its accessor finds the caller’s data handle acceptable.

This is true for any accessor type, not just for `aligned_accessor`. It is a design feature of `mdspan` that accessors can be stateless. Even if they have state, they do not need to be constructed with or store the data handle. As a result, an accessor might not see a data handle until `access` or `offset` is called. Both of those member functions are performance critical, so they cannot afford an extra branch on every call. Compare to `vector::operator[]`, which has preconditions but is not required to perform bounds checks. Using exceptions in the manner of `vector::at` could reduce performance and would also make `mdspan` unusable in a freestanding or no-exceptions context.

Note that `aligned_accessor` does not introduce any *additional* preconditions beyond those of the existing C++ Standard Library feature `assume_aligned`. In the words of one LEWG reviewer, `aligned_accessor` is not any more “pointy” than `assume_aligned`; it just passes the point through without “blunting” it.

Before submitting R0 of this paper, we considered an approach specific to `aligned_accessor`, that would wrap the pointer in a special data handle type. The data handle type would have an `explicit` constructor taking a raw pointer, with a precondition

that the raw pointer have sufficient alignment. The constructor would be explicit, because it would have a precondition. This design would force the precondition back to `mdspan` construction time. Users would have to construct the `mdspan` like this.

```
element_type* raw_pointer = get_pointer_from_somewhere();
using acc_type = aligned_accessor<element_type, byte_alignment>;
mdspan x{acc_type::data_handle_type{raw_pointer}, mapping,
acc_type{}};
```

We rejected this approach in favor of `is_sufficiently_aligned` for the following reasons. First, wrapping the pointer in a custom data handle class would make every access or offset call need to reach through the data handle’s interface, instead of just taking the raw pointer directly. The access function, and to some extent also offset, need to be as fast as possible. Their performance depends on compilers being able to optimize through function calls. The authors of `mdspan` carefully balanced generality with function call depth and other code complexity factors that may hinder compilers from optimizing. Performance of `aligned_accessor` matters as much or even more than performance of `default_accessor`, because `aligned_accessor` exists to communicate optimization potential. Second, the alignment precondition would still exist. Requiring the data handle type to throw an exception if the pointer is not sufficiently aligned would make `mdspan` unusable in a freestanding or no-exceptions context. Third, users should not have to pay for unneeded checks. The two examples in the wording express the two most common cases. If users get a pointer from a function like `aligned_alloc`, then they already know its alignment, because they asked for it. If users are computing alignment at run time to dispatch to a more optimized code path, then they know alignment before dispatch. In both cases, users already know the alignment before constructing the `mdspan`.

An LEWG poll on 2023/10/10, “[b]lock `aligned_accessor` progressing until we have a way of checking alignment requirements during `mdspan` construction,” resulted in no consensus. Attendance was 14.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
0	1	1	2	2

LEWG expressed an (unpolled) interest that we explore `mdspan` safety in subsequent work after the fall 2023 Kona WG21 meeting. LEWG recognized that this is a general issue with `mdspan` and asked us to explore safety in a way that is not specific to `aligned_accessor`.

§ 6 Implementation

We have tested an implementation of this proposal with the [reference `mdspan` implementation](#).

§ 7 Example


```

template<size_t byte_alignment>
using aligned_mdspan =
    std::mdspan<float, std::dextents<int, 1>, std::layout_right,
std::aligned_accessor<float, byte_alignment>>;

// Interfaces that require 32-byte alignment,
// because they want to do 8-wide SIMD of float.
extern void vectorized_axpy(aligned_mdspan<32> y, float alpha,
aligned_mdspan<32> x);
extern float vectorized_norm(aligned_mdspan<32> y);

// Interfaces that require 16-byte alignment,
// because they want to do 4-wide SIMD of float.
extern void fill_x(aligned_mdspan<16> x);
extern void fill_y(aligned_mdspan<16> y);

// Helper functions for making overaligned array allocations.

template<class ElementType>
struct delete_raw {
    void operator()(ElementType* p) const {
        std::free(p);
    }
};

template<class ElementType>
using allocation = std::unique_ptr<ElementType[],
delete_raw<ElementType>>;

template<class ElementType, std::size_t byte_alignment>
allocation<ElementType> allocate_raw(const std::size_t
num_elements)
{
    const std::size_t num_bytes = num_elements *
sizeof(ElementType);
    void* ptr = std::aligned_alloc(byte_alignment, num_bytes);
    return {ptr, delete_raw<ElementType>{}};
}

float user_function(size_t num_elements, float alpha)
{
    // Code using the above two interfaces needs to allocate to the
max alignment.
    // Users could also query aligned_accessor::byte_alignment
// for the various interfaces and take the max.
    constexpr size_t max_byte_alignment = 32;
    auto x_alloc = allocate_raw<float, max_byte_alignment>
(num_elements);
    auto y_alloc = allocate_raw<float, max_byte_alignment>
(num_elements);

    aligned_mdspan<max_byte_alignment> x(x_alloc.get());
    aligned_mdspan<max_byte_alignment> y(y_alloc.get());

    fill_x(x); // automatic conversion from 32-byte aligned to 16-

```

```

byte aligned
    fill_y(y); // automatic conversion again

    vectorized_axpy(y, alpha, x);
    return vectorized_norm(y);
}

```

§ 8 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The  character is used to denote a placeholder section number which the editor shall determine.

In *[version.syn]*, add

```
#define __cpp_lib_aligned_accessor YYYYMMML // also in <mdspan>
```

Adjust the placeholder value as needed so as to denote this proposal's date of adoption.

To the Header *<mdspan>* synopsis **[mdspan.syn]**, after class `default_accessor` and before class `mdspan`, add the following.

```

// [mdspan.accessor.aligned], class template aligned_accessor
template<class ElementType, size_t byte_alignment>
    class aligned_accessor;

```

At the end of **[mdspan.accessor.default]** and before **[mdspan.mdspan]**, add all the material that follows.

§ 8.1 Add subsection **[mdspan.accessor.aligned]** with the following

 Class template `aligned_accessor` **[mdspan.accessor.aligned]**

.1 Overview **[mdspan.accessor.aligned.overview]**

```

template<class ElementType, size_t the_byte_alignment>
struct aligned_accessor {
    using offset_policy = default_accessor<ElementType>;
    using element_type = ElementType;
    using reference = ElementType&;
    using data_handle_type = ElementType*;

    static constexpr size_t byte_alignment = the_byte_alignment;

    constexpr aligned_accessor() noexcept = default;

    template<class OtherElementType, size_t other_byte_alignment>
        constexpr aligned_accessor(
            aligned_accessor<OtherElementType, other_byte_alignment>)

```

```

noexcept;

    template<class OtherElementType>
        explicit constexpr aligned_accessor(
            default_accessor<OtherElementType>) noexcept;

    constexpr operator default_accessor<element_type>() const {
        return {};
    }

    constexpr reference access(data_handle_type p, size_t i) const
noexcept;

    constexpr typename offset_policy::data_handle_type
        offset(data_handle_type p, size_t i) const noexcept;

    constexpr static bool is_sufficiently_aligned(data_handle_type
p);
};

```

1 *Mandates:*

- (1.1) — `byte_alignment` is a power of two, and
- (1.2) — `byte_alignment >= alignof(ElementType)` is true.

2 `aligned_accessor` meets the accessor policy requirements.

3 `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.

4 Each specialization of `aligned_accessor` is a trivially copyable type that models semiregular.

5 `[0, n)` is an accessible range for an object `p` of type `data_handle_type` and an object of type `aligned_accessor` if and only if `[p, p + n)` is a valid range.

§ 8.2 Members [`mdspan.accessor.aligned.members`]

```

    template<class OtherElementType, size_t other_byte_alignment>
        constexpr aligned_accessor(
            aligned_accessor<OtherElementType, other_byte_alignment>)
noexcept {}

```

1 *Constraints:* `is_convertible_v<OtherElementType(*)[], element_type(*)[>` is true.

2 *Mandates:* `gcd(other_byte_alignment, byte_alignment) == byte_alignment` is true.

```

    template<class OtherElementType>
        explicit constexpr aligned_accessor(
            default_accessor<OtherElementType>) noexcept {};

```

- 3 *Constraints:* `is_convertible_v<OtherElementType(*)[], element_type(*)[]>` is true.

```
constexpr reference access(data_handle_type p, size_t i) const noexcept;
```

- 4 *Preconditions:* `p` points to an object `x` of a type similar (**[conv.qual]**) to `element_type`, where `x` has alignment `byte_alignment` (**[basic.align]**).

- 5 *Effects:* Equivalent to: `return assume_aligned<byte_alignment>(p)[i];`

```
constexpr typename offset_policy::data_handle_type  
offset(data_handle_type p, size_t i) const noexcept;
```

- 6 *Preconditions:* `p` points to an object `x` of a type similar (**[conv.qual]**) to `element_type`, where `x` has alignment `byte_alignment` (**[basic.align]**).

- 7 *Effects:* Equivalent to: `return p + i;`

```
constexpr static bool is_sufficiently_aligned(data_handle_type p);
```

- 8 *Preconditions:* `p` points to an object `x` of a type similar (**[conv.qual]**) to `element_type`.

- 9 *Returns:* true if `x` has alignment at least `byte_alignment`, else false.

[Example: The following function `compute` uses `is_sufficiently_aligned` to check whether a given `mdspan` with `default_accessor` has a data handle with sufficient alignment to be used with `aligned_accessor<float, 4 * sizeof(float)>`. If so, the function dispatches to a function `compute_using_fourfold_overalignment` that requires fourfold overalignment of arrays, but can therefore use hardware-specific instructions, such as four-wide SIMD (Single Instruction Multiple Data) instructions. Otherwise, `compute` dispatches to a possibly less optimized function `compute_without_requiring_overalignment` that has no overalignment requirement.

```
extern void  
compute_using_fourfold_overalignment(  
    std::mdspan<float, std::dextents<size_t, 1>, std::layout_right,  
    std::aligned_accessor<float, 4 * alignof(float)>> x);  
  
extern void  
compute_without_requiring_overalignment(  
    std::mdspan<float, std::dextents<size_t, 1>> x);  
  
void compute(std::mdspan<float, std::dextents<size_t, 1>> x)  
{  
    auto accessor = aligned_accessor<float, 4 * sizeof(float)>{};  
    auto x_handle = x.data_handle();  
  
    if (accessor.is_sufficiently_aligned(x_handle)) {  
        compute_using_fourfold_overalignment(mdspan{x_handle,  
x.mapping(), accessor});  
    }  
}
```

```

        else {
            compute_without_requiring_overalignment(x);
        }
    }
}

```

–end example]

[Example: The following example shows how users can fulfill the preconditions of `aligned_accessor` by using existing C++ Standard Library functionality to create overaligned allocations. First, the `allocate_overaligned` helper function uses `aligned_alloc` to create an overaligned allocation.

```

template<class ElementType>
struct delete_with_free {
    void operator()(ElementType* p) const {
        std::free(p);
    }
};

template<class ElementType>
using allocation = std::unique_ptr<ElementType[],
delete_with_free<ElementType>>;

template<class ElementType, size_t byte_alignment>
allocation<ElementType> allocate_overaligned(const size_t
num_elements)
{
    const size_t num_bytes = num_elements * sizeof(ElementType);
    void* ptr = std::aligned_alloc(byte_alignment, num_bytes);
    return {ptr, delete_with_free<ElementType>{}};
}

```

Second, this example presumes that some third-party library provides functions requiring arrays of `float` to have 32-byte alignment.

```

template<size_t byte_alignment>
using aligned_mdspan = std::mdspan<float, std::dextents<int, 1>,
std::layout_right, std::aligned_accessor<float,
byte_alignment>>;

extern void vectorized_axpy(aligned_mdspan<32> y, float alpha,
aligned_mdspan<32> x);
extern float vectorized_norm(aligned_mdspan<32> y);

```

Third and finally, the user’s function `user_function` would begin by allocating “raw” overaligned arrays with `allocate_overaligned`. It would then create aligned `mdspan` with them, and pass the resulting `mdspan` into the third-party library’s functions.

```

float user_function(size_t num_elements, float alpha)
{
    constexpr size_t max_byte_alignment = 32;
    auto x_alloc = allocate_overaligned<float, max_byte_alignment>
(num_elements);
    auto y_alloc = allocate_overaligned<float, max_byte_alignment>

```

```
(num_elements);

    aligned_mdspan<max_byte_alignment> x(x_alloc.get());
    aligned_mdspan<max_byte_alignment> y(y_alloc.get());

    // ... fill the elements of x and y ...

    vectorized_axpy(y, alpha, x);
    return vectorized_norm(y);
}
```

–end example]